

PRE-WORKSHOP

While You Wait: Setup Checklist

1

Confirm Your Toolkit

Open GitHub Desktop — sign in.
Open VS Code.
If anything's missing, flag a mentor now.

2

Create a Replit Account

Go to replit.com
Sign up with your GitHub email.
Connect to GitHub:
Account → Connected Services → GitHub → Authorize

3

Partner Up

Sit together. You'll both build AND edit today — no one watches from the sidelines.

 Step 2 is the critical path. If Replit and GitHub aren't connected before we start, you'll spend Round 1 troubleshooting instead of building.

GitHub 2.0: The Handoff

Building Together Without Breaking Things

Aaron McKeever — The Upskilling Labs

PART 1

The Map for Today

Reviewing What Powers the Handoff

~12 minutes

The Rule Change

Workshop 1

We got familiar with GitHub terminology and committed .md files directly to main; however, that was solo practice on a private repository.



From Now On

1. Main should always be protected!
 2. Every change will go through a branch and a Pull Request
- These are best for multiple people working on a shared repo.*

✓ Now we practice the teamwork. Treat Main as special — today and for the entire build cycle.

The Branch Workflow

Review from Workshop 1, Part 4

- 1 Main is the “official” version. Nobody works directly on main.
- 2 New task → new branch. Name it descriptively: feature/new-headline, fix/broken-link
- 3 Do your work on the branch. Commit often.
- 4 When ready, open a Pull Request on GitHub.
- 5 A teammate reviews, then merges.
- 6 Delete the branch as best practice (optional)



New today: Steps 4 and 5 — opening and reviewing a Pull Request — are the skills we’re adding.

Typical Daily Cycle + The Handoff Layer

Review from Workshop 1, Part 6 — with a new dimension

Your Daily Cycle (recap)

Start: Fetch/Pull → Check your branch

Work: Make changes in your editor

End: Commit → Push before you close your laptop

The Handoff Layer (new)

Before you start: Pull to get your teammate's latest

Before you push: Confirm you're on YOUR branch

After you push: Open a Pull Request so your partner can review

The cycle runs between people, not just between your machine and GitHub.

AI and the Repo

Review from Workshop 1, Part 5

The prompt is part of the project materials

/prompts folder — one markdown file per significant generation

Commit messages carry context: AI-generated vs. hand-edited (advantages and drawbacks to each)

Why This Matters in 20 Minutes

Because you'll receive your partner's AI-built code but didn't write it nor prompt it, *a prompt doc can be an important!*

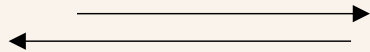


You will want to discuss a manageable approach to adding and naming prompts. These should be larger scale prompts that begin the feature building process. Chat histories can also be shared as a link with team members if that context is helpful for difficult features.

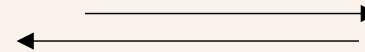
Today's Loop

We'll come back to this slide between every round.

Builder



GITHUB
(Hub)



Editor

Build in Replit → Push to GitHub → Partner clones, branches, edits → Opens PR → You review & merge → Pull back

— AND AT THE SAME TIME, THE REVERSE —

Partner builds → Pushes to GitHub → You clone, branch, edit → Open PR → Partner reviews & merges → Pulls back

✓ **Goal:** Everyone leaves feeling comfortable pushing to and pulling from a project.

PART 2

Watch One Full Loop

Overview of Live Demo and Questions

~10 minutes

1

DEMO — Phase 1: Build and Ship

- Paste Prompt 1 (filled in) into Replit
- Replit Agent builds the page
- Git pane: Stage → Commit → Push to GitHub
- On GitHub.com: create prompts/scaffold-prompt.md
- Settings → Collaborators → invite partner

HANDOFF ONE: *"My work is now visible to my partner."*

2

DEMO — Phase 2: Clone, Branch, and Edit

- Clone in GitHub Desktop (File → Clone)
- IMMEDIATELY create a branch: feature/demo-edit
- Confirm the branch indicator — NOT main
- Project-wide search (Ctrl/Cmd+Shift+F) to find the headline
- Make one change, commit with context, push the branch

“Before you change anything — check your branch.”

3

DEMO — Phase 3: Propose, Review, Merge

→ GitHub.com: yellow banner → “Compare & pull request”

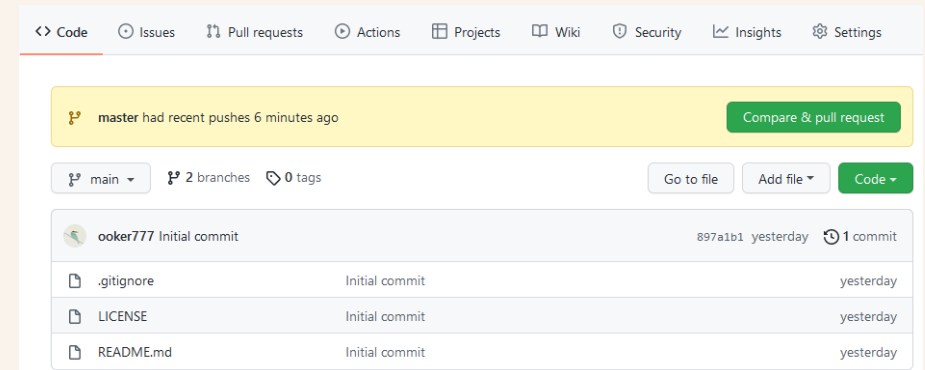
→ Title it clearly, describe the change

→ Partner opens the PR → “Files changed”

→ Green = added. Red = removed.

→ Leave a comment → Merge → Delete branch

→ In Replit: Git pane → Pull



Questions?

PART 3

Build and Ship

Everyone Builds, Everyone Ships

~15 minutes



Both of you build at the same time, on your own Replit accounts. Sit together, help each other troubleshoot!

Prompt 1: Fill in the Blanks

Build a single-page personal website. Keep it SIMPLE:
one page, no routing, no backend, no forms.

The page is for: _____ Headline: " _____ "
One sentence about me: " _____ "
Three Things About Me cards: _____, _____, _____
Color mood: _____

Footer: "Built at The Upskilling Labs — GitHub 2.0 Workshop"

Keep all text easy to find and edit in the code.
No extra pages, animations, or heavy libraries.



Fill in your blanks, paste into Replit, let it build. Good enough is good enough — do NOT iterate.

Ship It: Your Checklist

1

Build

Replit Agent builds your page.
Click around the preview.
Good enough!

2

Push

Git pane → Stage all →
Commit → Push.
Repo: <yourname>-handoff-
lab
For Replit: **Must be PUBLIC.**

3

Confirm

Check GitHub.com.
See your code on main?

4

Prompt Doc

GitHub.com: Add file →
prompts/scaffold-prompt.md
→ paste prompt → commit.

5

Invite

Settings → Collaborators →
add your partner's
GitHub username.

✓ Thumbs up when BOTH partners have a live repo on GitHub with a prompt doc committed and a collaborator invite sent.

Also: Accept your partner's invitation to their repo. Check email or github.com/notifications.

PART 4

Clone, Branch, and Change

Everyone Edits Their Partner's Code

~20 minutes



You're about to open code you didn't write, built by an AI you didn't prompt. This is the most common situation in modern team development.

Your Mission as the Editor

What Just Happened

Your partner built a page with AI and shipped it to GitHub

You've been invited as a collaborator on their repo

You didn't write this code and didn't prompt it

This is normal — it's how teams work

Your Toolkit

`prompts/scaffold-prompt.md`
tells you what was built and why

Ctrl/Cmd+Shift+F
project-wide search finds anything

You don't need to understand every line — just find the piece you're changing

Steps 1–4: Get Set Up

1

Accept Invite

Check email or GitHub notifications for your partner's repo.

2

Clone

GitHub Desktop:
File → Clone →
find your partner's repo.

3

Branch FIRST

Current Branch →
New Branch →
feature/<yourname>-edit

4

Read the Map

Open scaffold-prompt.md in VS Code.
Understand what was built. Copy and
paste snippets into your AI chat if you
need help.

✓ Before you touch any code: check the branch indicator in GitHub Desktop. If it says “main” — STOP. Create your branch first.

Secret Help: github.dev

If Your Local Setup Breaks

Open any repo on GitHub.com → press the `.` key → full VS Code opens in the browser.
Edit, commit, push. No install needed.

Why Teams Use This

- Working from a borrowed computer or a machine without your tools
- Quick edits when you're away from your main setup
- Reviewing a teammate's branch in a real editor (better than GitHub's diff view)
- Getting a new team member productive before they've set up locally

The Change Menu — Pick ONE

★ Text Swap

Use project-wide search to find the headline text. Change it to something new. Save.

★★ Add a Fourth Card

Find a “Three Things” card in the code. Copy that entire code block, paste below, edit the text.

★★★ AI Color Change

Find color hex values. Ask your AI assistant how to shift the palette. Save the exchange as prompts/edit-1.md.

💡 Finishing one clean change matters more than attempting something ambitious. Option 3 teaches using AI as a reading aid for inherited code, which is an important skill to learn!

Steps 5–8: Ship Your Change

1

Edit

Make your change in VS Code.
Save (Ctrl/Cmd+S).

2

Commit

GitHub Desktop:
write a message with context.
E.g., “Change headline —
hand-edited”

3

Push

Click “Publish branch”
to push your branch
to GitHub.

4

Open PR

GitHub.com → yellow banner →
“Compare & pull request.”
Title + description.

✓ Thumbs up when you see the yellow “Compare & pull request” banner on GitHub.com.

Opening a Pull Request

The skill Workshop 1 described but never showed you

1. You'll see a yellow banner on GitHub.com. Click "Compare & pull request."
2. Title: clear, short description. Example: "Add fourth card to Three Things section."
3. Description: one sentence — what you changed and why. Mention if AI-assisted.
4. Check the arrow: feature/<yourname>-edit → main (your branch INTO main).
5. Click "Create pull request."



A Pull Request says: "Here's what I'd like to change — look before it goes live." It's a proposal, not a command. This is how professional teams work.

PART 5

Close the Loop

Review, Merge, and Sync Back

~12 minutes



Switch back to YOUR repo — you've got a Pull Request waiting from your partner.

Reviewing a Pull Request

The second new skill from today

1. Open YOUR repo on GitHub.com. You should see your partner's PR.
2. Click the "Files changed" tab.
3. Green lines = added. Red lines = removed. This is the diff — the exact picture of what changes.
4. Click any line number to leave a comment. Even "Looks good!" counts.
5. Back on "Conversation" tab → "Merge pull request" → Confirm → "Delete branch."

💡 The "Files changed" view is the genius of the whole system. You see exactly what changes before they become official. No surprises, no overwritten work.

Merge & Watch the Sync

After Merging

1. The branch is deleted — repo is clean
2. Main now contains your partner's change
- 3. Open Replit → Git pane → Pull**
4. The change appears in your Replit project

Result

The code made a full round trip:

**Builder → GitHub → Partner → Branch → PR → Main →
Builder**

✓ Two complete handoff loops — one on each repo. That's the workflow your build-cycle team will run dozens of times.

PART 6

The Handoff Protocol

Reflection and Discussion of Moving from Workshop to Build Cycle

~10 minutes

THE HANDOFF PROTOCOL

The Upskilling Labs

1

Main should always works. No direct commits to main.

2

Branch first, then edit. Check the branch indicator before you type.

3

One branch = one person = one task. Name descriptively: feature/..., fix/...

4

Every AI generated feature gets a prompt doc in /prompts, committed with the code.

5

Commits say who/how. Hand-edited vs. AI-generated, and which prompt doc was used

6

All changes enter main through a PR. At least one teammate reviews before merging.

Pull before you start; push before you stop.

Before Your First Build-Cycle Session

1

Set Up the Team Repo

One teammate creates the repo and invites everyone.

2

Test the Loop

Everyone clones, pushes one test commit ON A BRANCH with a PR — your “hello world” handoff.

3

Plan Together

Write your scaffold prompt together. Commit to /prompts before anyone builds.

4

Agree on the Protocol

Read the Handoff Protocol as a team. Out loud. These are your house rules.

? Questions about next steps for your project?

Before You Close Your Laptop



Star both repos (yours and your partner's) on GitHub.com — click the star icon so you can find them in Your Stars later.



Bookmark the workshop materials folder (Google Drive link).



Confirm you can find your repos from your GitHub profile → Your Repositories.

Thanks, Upskillers.

Short anecdote: It took me a while to get used to Git and GitHub.

However, as you build with your team and run through the cycle dozens of times,
it will become second nature.

Just start make sure you start early and keep working with version control!

Aaron McKeever

Slack | Calendly | GitHub